Social Science Simulations (SVUH.00.300) · Seminar 6

# Monte Carlo Analysis and Python-based Simulations

Kishor Acharya · Institute of Social Studies, University of Tartu

# Three questions that are hard with pen and paper

### A referendum

Polls say 52% yes, but polls are never exact and turnout is uncertain. How likely is it to pass?

### A survey

You expect a modest difference between two groups. How many people must you interview to detect it?

### A city

Each resident has only a mild preference for similar neighbours. Does the city become segregated?

# Today: a ladder of six stops, and two tabs to keep open

1. **Two dice**
   estimate a probability by counting
2. **Darts and π**
   measure a number with randomness
3. **A referendum**
   uncertainty in, a range of outcomes out
4. **Planning a survey**
   test a study design on fake data
5. **Schelling's city**
   run an agent-based model many times
6. **Where this leads**
   bootstrap, sensitivity, MCMC

### Tab 1: animations

acharyakishor.com/simulations
Drag sliders, no code.

### Tab 2: notebook

Colab link on the course page.
Same experiments, with the code.

*At every stop an "On your screen" slide tells you which animation and which notebook section to open, what each slider means, and what to look for.*

Where the name comes from

# Solitaire, a casino, and a secret project

Los Alamos, 1946

# 1946: a sickbed and a deck of cards

**Stanisław Ulam**



Los Alamos National Laboratory, 1940s · public domain

- Stanisław Ulam, recovering from a serious illness, plays Canfield solitaire to pass the time.
- He wonders: what is the chance that a randomly dealt layout can be won?
- He tries to calculate it by pure combinatorics. Hopeless.
- Then a simpler idea: lay the cards out a hundred times, count how many games come out, and take the proportion as the answer.

*Metropolis, N. (1987). The beginning of the Monte Carlo method. Los Alamos Science, 15, 125–130.*

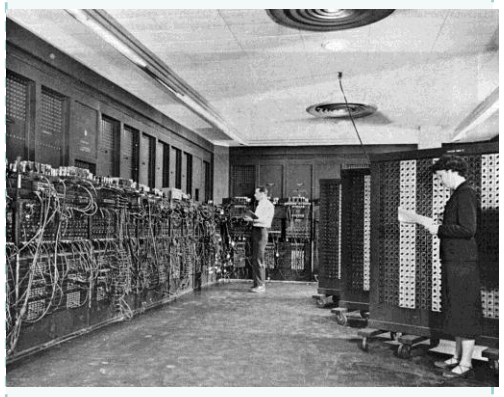
# The same trick for a much harder problem

**John von Neumann**



Los Alamos National Laboratory · public domain

**ENIAC (1947)**



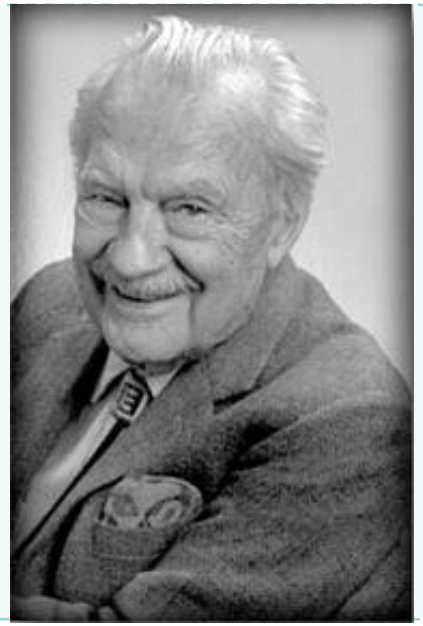
U.S. Army photo · public domain

- At Los Alamos, physicists could not solve the equations for how neutrons travel through material.
- But following one neutron on its random path is easy: how far it goes, whether it is absorbed or scattered.
- Follow thousands of neutrons and count: the answer the equations could not give.
- John von Neumann worked out how to run it on ENIAC, the first electronic computer, in 1947.

***Trade thinking for computing***

# Why it is called Monte Carlo

**Nicholas Metropolis**



**Casino de Monte-Carlo**



- The work was secret and needed a code name.
- Nicholas Metropolis proposed “Monte Carlo”: the casino in Monaco where Ulam's uncle used to borrow money from relatives to gamble.
- First public description: Metropolis & Ulam (1949).

*Metropolis, N., & Ulam, S. (1949). The Monte Carlo method. Journal of the American Statistical Association, 44(247), 335–341.*

# Three things to keep from the story

### Born from gambling

Cards and dice are the simplest random processes. That is why we start with them, and why the vocabulary (odds, draws, runs) comes from the casino.

### Thinking for computing

When the formula is out of reach, simulate the process and let the machine count. Patience is something a computer has in unlimited supply.

### Impossible before computers

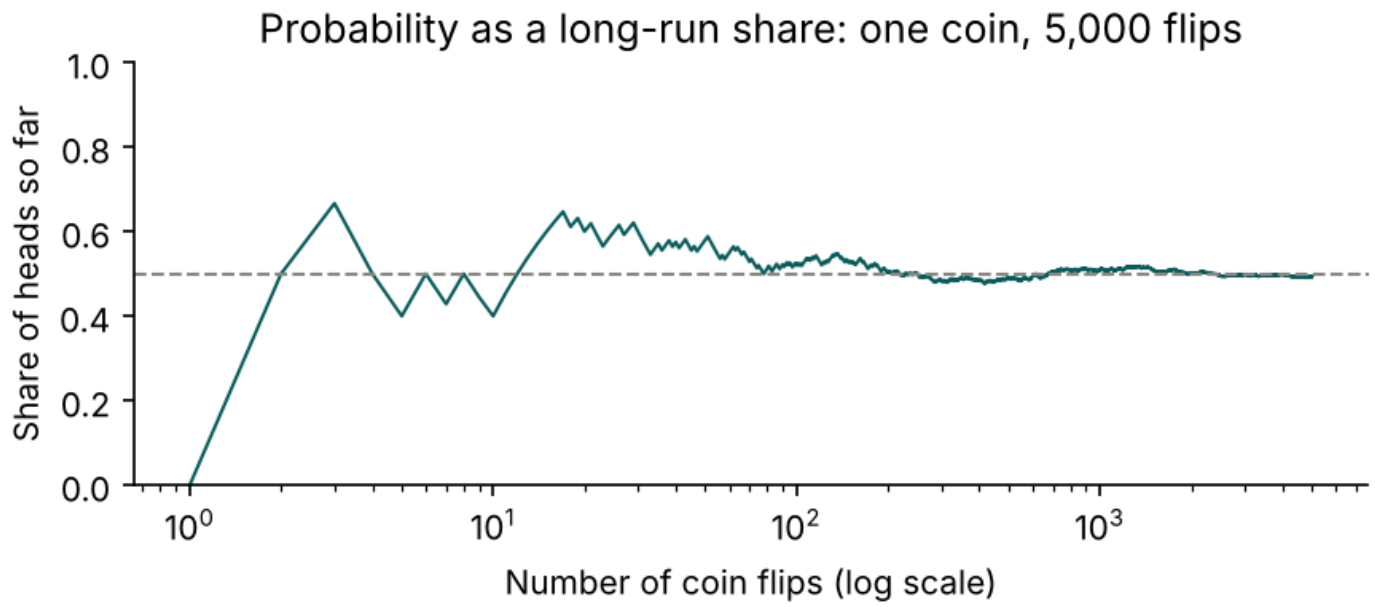
Buffon dropped needles by hand in 1777 to estimate π. What was new in 1946 was scale, and the realisation that almost any problem with randomness in it can be attacked this way.

Part A · Four ideas the method rests on

# Probability, random numbers, distributions, and the law of large numbers

Notebook Part A follows the same four headings.

# Probability is a long-run share




- "Probability of heads is ½" means: in an enormous number of flips, about half are heads.
- After 10 flips the share might be 0.3 or 0.7. After 5,000 it is very close to 0.5.
- Probability is where the line settles.

**So a probability can be measured by repeating the experiment and taking the share. No formula, only patience.**

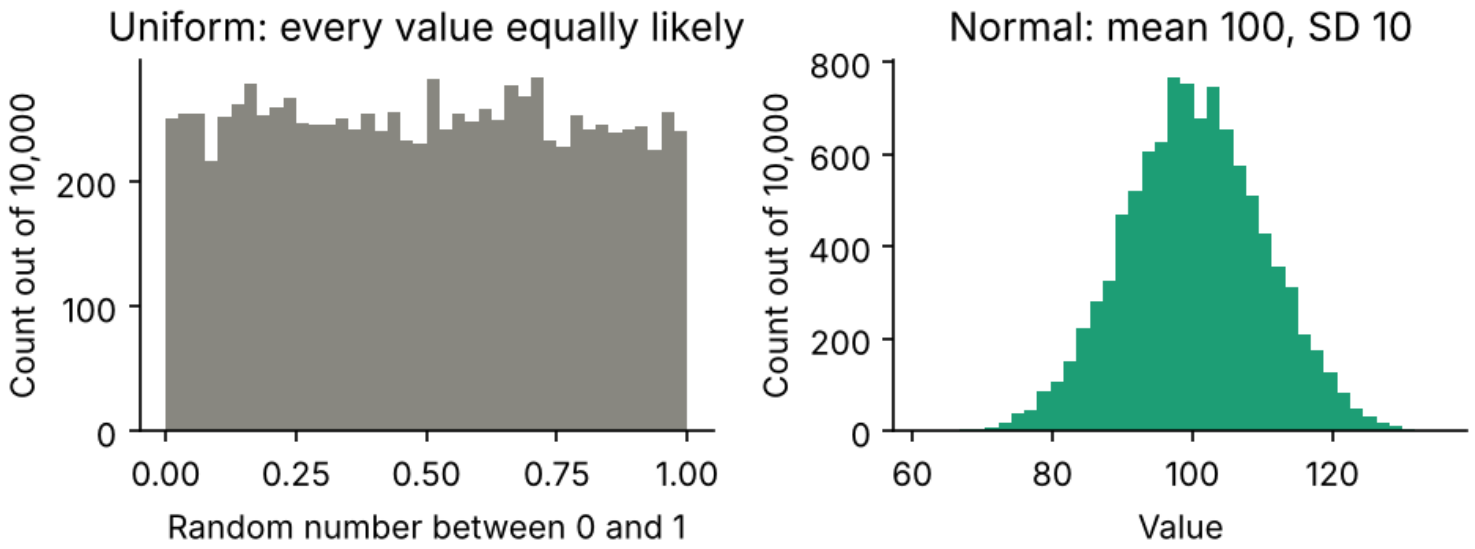
*Notebook A.1: drag "Flips" from 10 to 5,000 and watch the swings narrow.*

# Random numbers from a computer

| What you want | Excel | Python (notebook) |
|---|---|---|
| a number between 0 and 1, all equally likely | `=RAND()` | `rng.random()` |
| a whole number from 1 to 6 (a die) | `=RANDBETWEEN(1,6)` | `rng.integers(1, 7)` |
| a bell-curve number, mean 100, SD 10 | `=NORMINV(RAND(),100,10)` | `rng.normal(100, 10)` |

- A computer cannot produce truly random numbers. It runs a formula whose output passes every test of randomness: pseudo-random. Good enough for us.
- The sequence starts from a seed. Same seed, same “random” numbers, every time.
- That is a feature: your colleague can reproduce your simulation exactly. Every notebook section begins by fixing a seed.

# Distributions: the shape of uncertainty




*Notebook A.3: move the mean and SD sliders; the shaded band always holds about 95% of the draws.*

- A distribution says which values a random quantity takes, and how often. Draw many values, make a histogram, and you can see it.
- Uniform: every value in a range equally likely. The computer's default.
- Normal (bell curve): values cluster around a mean; the standard deviation (SD) sets the spread. 68% within 1 SD, 95% within 2, 99.7% within 3.

# The law of large numbers, and the $1/\sqrt{N}$ rule

**1713**

Jacob Bernoulli proves it: the average of many random trials settles on the true value

**$1/\sqrt{N}$**

the typical error of a Monte Carlo estimate shrinks with the square root of the number of runs

**100× → 10×**

a hundred times more runs buys ten times less error

- This is the guarantee that Monte Carlo works at all.
- It also tells us the price: diminishing returns. Ten thousand runs give roughly two decimal places; a million give three.
- Consequence: always report a Monte Carlo result together with its spread, and run at least a thousand times.

*Bernoulli, J. (1713). Ars Conjectandi.*

# Part B · The recipe

**Monte Carlo answers a question by generating many random versions of the situation and counting what happens.**

## Three ingredients

1. A model with something uncertain in it
2. A random number generator to draw one version
3. Repetition and aggregation: thousands of draws, then count or average

## Four steps

1. Define the output you care about
2. Describe each uncertain input by a distribution
3. Draw, run the model once, record. Repeat N times
4. Summarise the N outputs, with their uncertainty

Stop 1

# Two dice, and the first Monte Carlo

Estimate a probability by counting. Watch the error shrink.

# Roll two dice and add them. How often is the sum 7?

### The exact answer

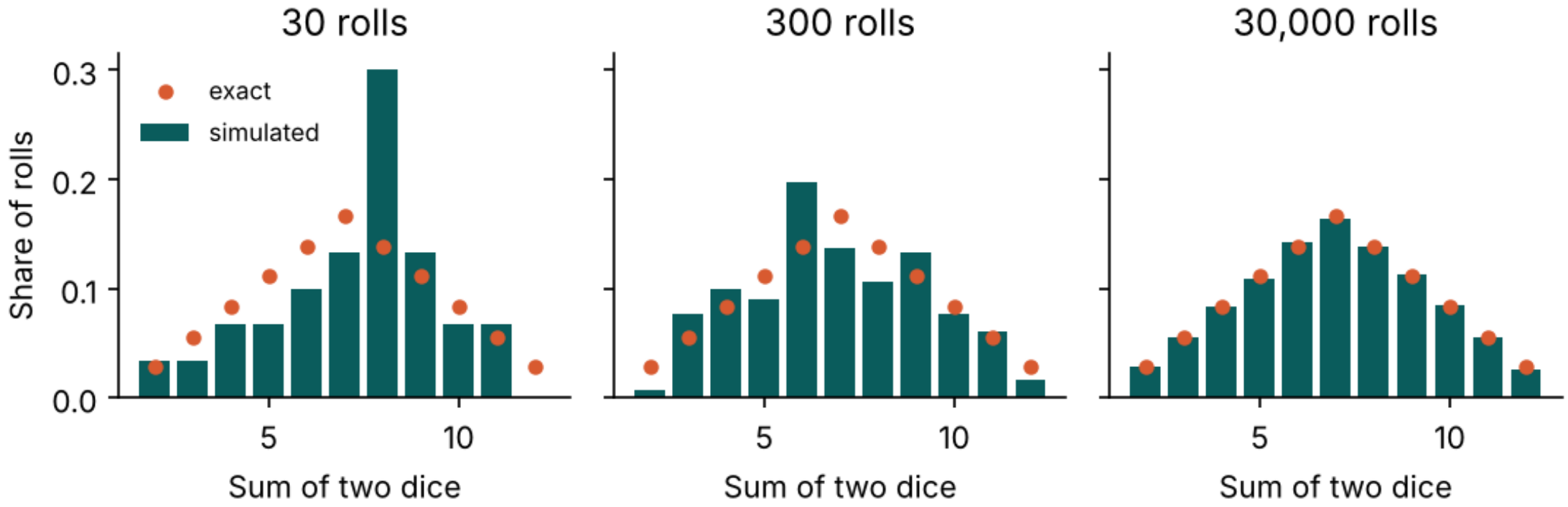
6 × 6 = 36 equally likely pairs of faces.
Six of them add to 7: 1+6, 2+5, 3+4, 4+3, 5+2, 6+1.
Probability = 6/36 = 1/6 ≈ 16.7%.

### The Monte Carlo answer

Roll two dice 10,000 times. Add. Count how many sums are 7. Divide by 10,000.
Result: about 0.167, give or take a few thousandths.

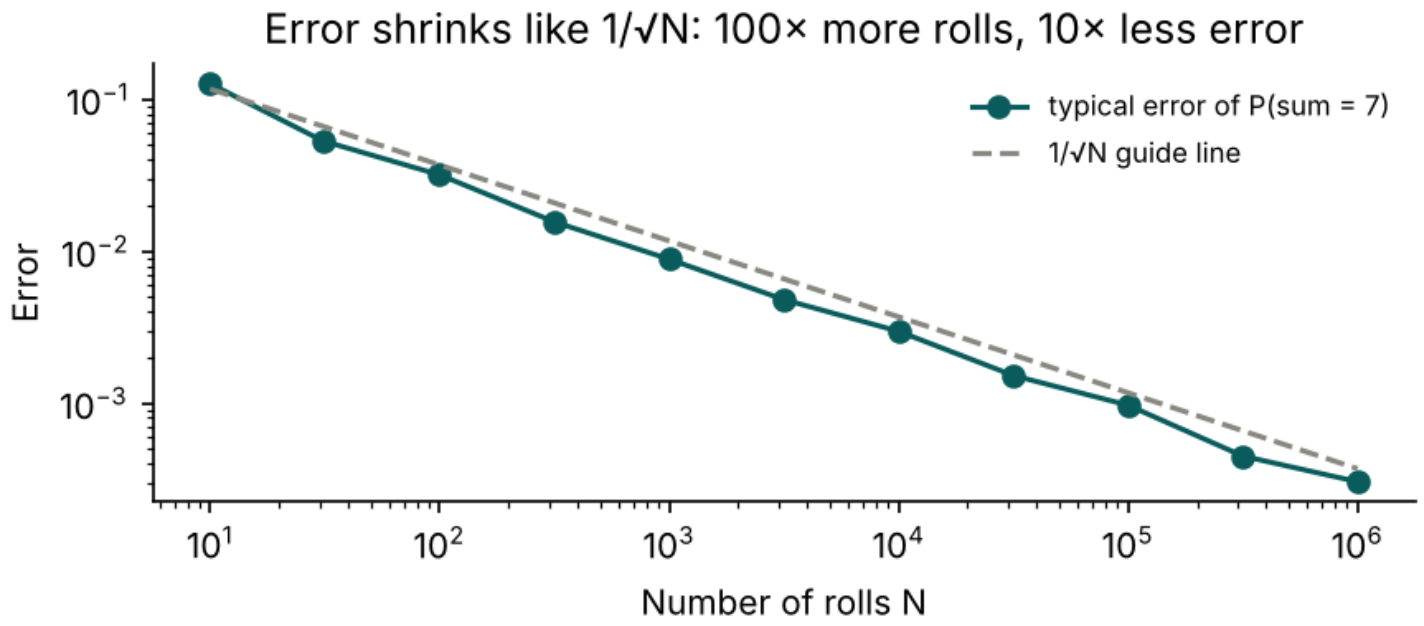
*It does not know any of the left box. It rolls, adds, and counts. (Notebook 1.1: four lines of code.)*

# More rolls, better picture




*Bars: simulated share of each sum. Dots: the exact probabilities. With 30 rolls the shape is barely there; with 30,000 the bars sit on the dots.*

# The error shrinks like 1/√N




- Each point: the typical error of the estimate of P(sum = 7), averaged over 20 repetitions.
- On log-log axes the points fall on a straight line with slope −½.
- Every hundredfold increase in rolls buys a tenfold reduction in error.

# Stop 1 · On your screen

**Website animation** “Two dice”: drag Rolls, press Roll again several times at each setting.

**Notebook** sections 1.1 to 1.3: run each cell; the ▶ Try it cell has the slider.

| Parameter | What it means | What to watch |
|---|---|---|
| **Rolls (N)** | How many times the two dice are thrown | Bars approach the dots as N grows; the error on P(7) shrinks roughly as 1/√N |
| **Roll again** | A fresh set of random rolls at the same N | At small N each press gives a visibly different picture: the estimate is itself random |

✎ **Exercise, for later (notebook):** estimate the probability of a double (both dice equal); exact answer 1/6.

# What we learned at stop 1

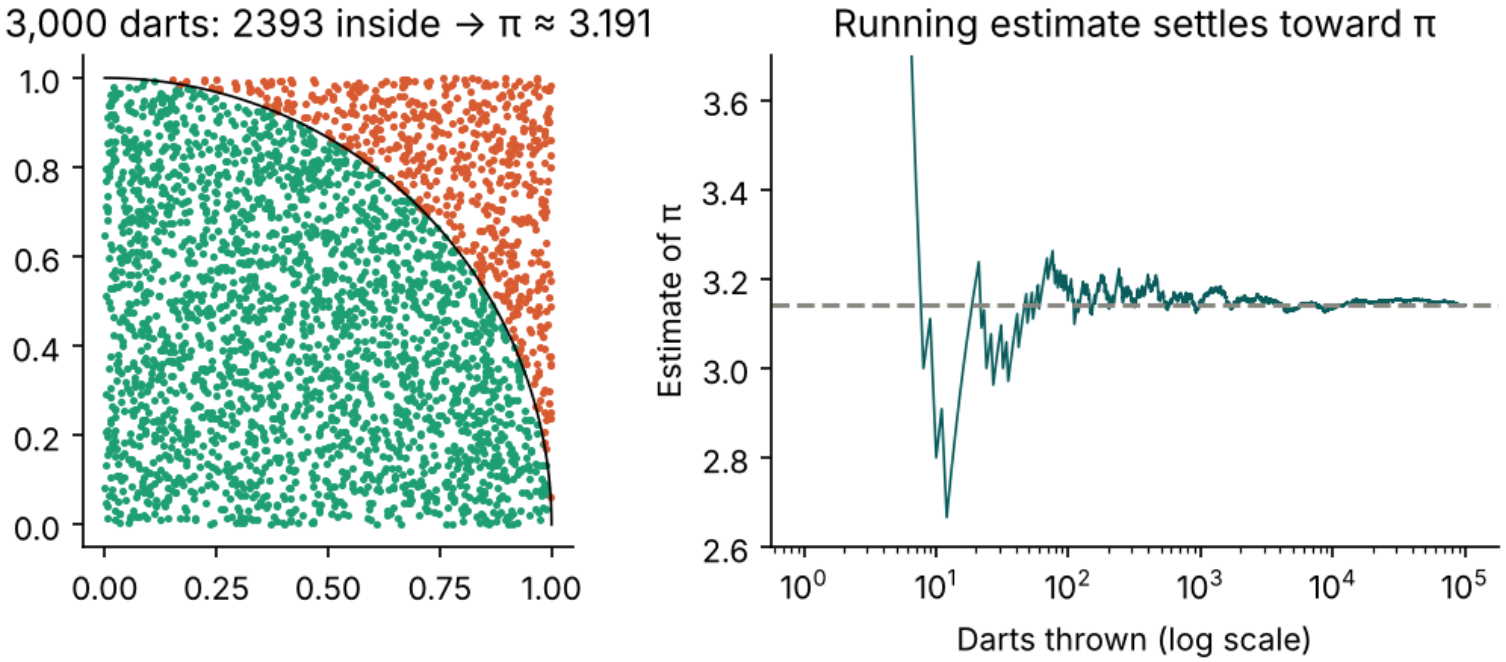
- A probability can be measured by counting, without any formula.
- The estimate is itself random: run it twice and you get two slightly different numbers.
- More runs give a better estimate, with diminishing returns ($1/\sqrt{N}$).

Stop 2

# Darts, π, and measuring with randomness

Now the output is a number, not a probability.

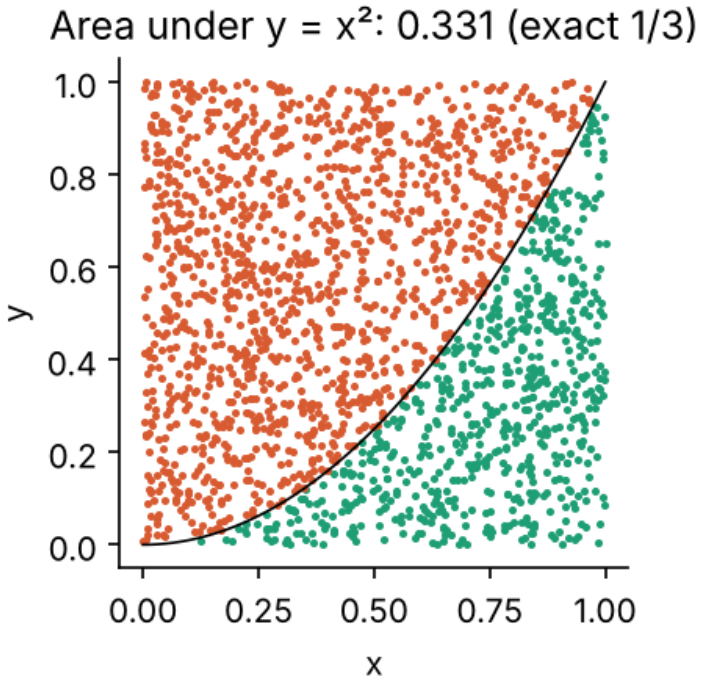
# Throw darts; count the ones inside the circle




- Square of side 1: area 1. Quarter circle inside it: area π/4 ≈ 0.785.
- Throw darts completely at random. The share landing inside equals the share of the area.
- π ≈ 4 × (inside ÷ thrown)

***Nobody is gambling here. Randomness is a ruler.***

# Any area, in any number of dimensions




**Comte de Buffon**



- Replace the circle by the region under any curve: the count gives its area, the integral. Here $y = x^2$, exact area 1/3.
- For one curve a calculator does better. In ten or a hundred dimensions ordinary methods become impossibly slow; random sampling costs the same regardless of dimension.
- Buffon's needle (1777): drop needles on a boarded floor; the share crossing a crack depends on π. The first Monte Carlo, done by hand.

*Buffon, G.-L. L. (1777). Essai d'arithmétique morale. · Verschuuren (2017), simulation 89, builds the same example in Excel.*

# Stop 2 · On your screen

**Website animation** “Darts and π”: drag Darts, press Throw again.

**Notebook** sections 2.1 to 2.3, plus the Buffon's needle cell.

| Parameter | What it means | What to watch |
|---|---|---|
| **Darts (N)** | How many random points are thrown at the square | The running estimate wanders, then settles near 3.14 but never becomes exact |
| **Throw again** | A fresh set of darts at the same N | Two runs at the same N give two different paths and two different final estimates |
| **f(x) (notebook 2.3)** | The curve whose area is measured | Change x**2 to any curve and the same count gives its area |

✎ **Exercise, for later (notebook):** how many darts are needed before the estimate is reliably within 0.01 of π? Does $1/\sqrt{N}$ predict it?

# What we learned at stop 2

- Monte Carlo can estimate numbers, not just probabilities, by turning a measurement into a count.
- The running estimate converges but never becomes exact: always report a margin.
- The method does not care how many dimensions the problem has. That is its great advantage.

Stop 3

# A referendum, and taming the future

Feed distributions in. Read a distribution out.

# Polls say 52% yes. Will it pass?

**52%**

the polls say yes

**± 3 points**

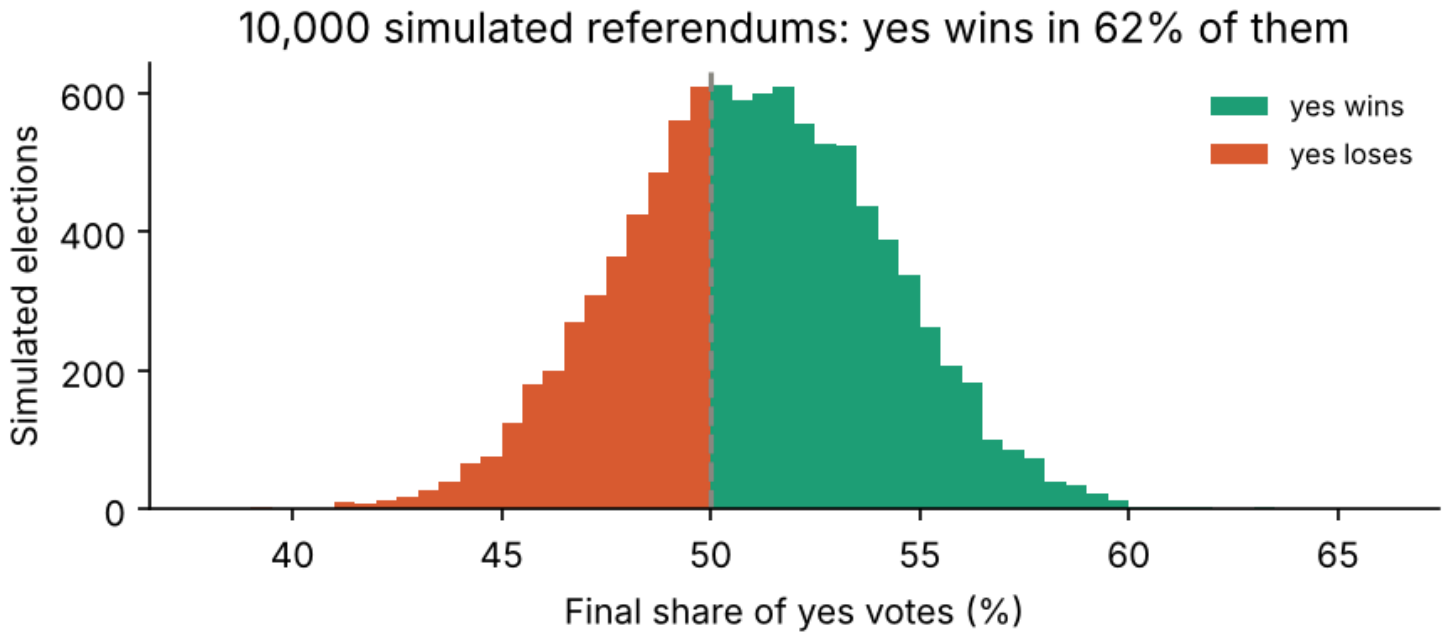
polls have historically missed the result by about this much, either way

**−1 ± 1**

the yes side tends to turn out slightly less, costing about a point

- A single number cannot answer this. 52 − 1 = 51, “yes, narrowly”, ignores the ±3 and the ±1.
- The honest answer is a probability. Monte Carlo gets it by simulating ten thousand election nights, each with its own random poll error and turnout effect.

# Ten thousand election nights




**62%**

chance the proposal passes

**46% – 56%**

90% of outcomes fall in this range

Three things the number 52 could not say: it will probably pass; there is a real chance (38%) it fails; almost any result between 46 and 56 would be unsurprising.

*Verschuuren, G. M. (2017). 100 Excel Simulations: "taming the future" with NORMINV.*

# Stop 3 · On your screen

**Website animation** “Referendum”: three sliders for the inputs, one for the number of simulations.

**Notebook** sections 3.1 and 3.2, the ▶ Try it cell, and the budget exercise skeleton.

| Parameter | What it means | What to watch |
|---|---|---|
| **Polled yes %** | What the last polls say | Move it toward 50: the pass probability falls fast near the line |
| **Poll error ±** | How wrong polls usually are (SD, points) | Widen it: the histogram flattens and the probability moves toward 50/50 |
| **Turnout tilt** | Average points gained or lost through turnout | Shifts the whole histogram left or right |
| **Simulations** | How many election nights are played | At 100 the histogram is ragged; at 10,000 it is smooth (1/√N again) |

✎ **Exercise, for later (notebook):** the fieldwork budget: three uncertain cost items, find the probability of exceeding €12,000, then the budget that is safe 95% of the time.

# What we learned at stop 3

- Replace every fixed input by a distribution; the output becomes a distribution too.
- Summarise it as a probability, a median, and a range (for example the 5th to 95th percentile).
- The histogram is more honest, and more useful, than any single number.

# Five-minute break

Next: testing a study design before collecting any data, then Schelling's city.

Stop 4

# Planning a survey by simulating fake data

Invent a world where you know the truth. Check whether your analysis finds it.

# How many people do I need?

Statistical power: the chance a study detects an effect that is really there. The usual target is 80%.

1

Invent a world in which the difference is real, of a given size

2

Simulate one fake study: a sample from each group, run your test, note whether $p < 0.05$

3

Repeat 1,000 times. The share that detected the effect is the power

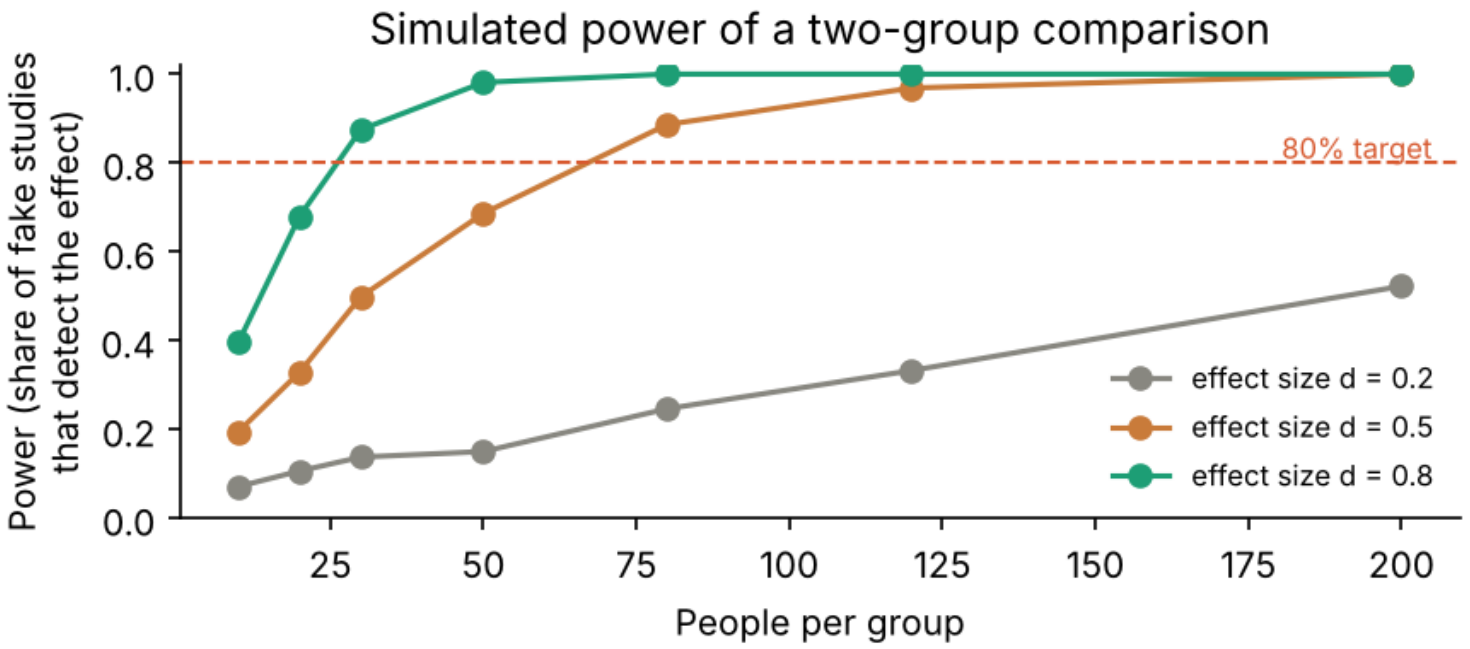
4

Change the sample size until the power reaches 80%

This is the recipe of Part B with your own statistical test sitting inside the model. There are formulas for simple cases; the simulation works for any design, however complicated.

*Effect sizes in SD units follow Cohen, J. (1988). Statistical Power Analysis for the Behavioral Sciences.*

# Power by simulation




- Effect sizes in standard-deviation units: small 0.2, medium 0.5, large 0.8.
- A medium effect needs about 64 people per group for 80% power.
- A small effect needs about 400. This is why underpowered studies of small effects are so common.

*The same trick checks whether any method behaves as advertised: is my estimator unbiased? Does my 95% interval really contain the truth 95% of the time? (Notebook 4.3.)*

# Stop 4 · On your screen

**Website animation** “Power”: two sliders, a bar that turns green at 80%.

**Notebook** sections 4.1 to 4.3 (4.3 is the coverage check).

| Parameter | What it means | What to watch |
|---|---|---|
| **People per group** | Sample size of each of the two groups | Power rises with n but flattens: going from 100 to 200 buys little for a medium effect |
| **Effect size (d)** | How big the real difference is, in SD units | Small effects need very large samples; at d = 0.2 the bar stays red until several hundred per group |

✎ **Exercise, for later (notebook):** an unequal design (40 vs 20): modify power() to take two sample sizes and find how many are needed in the smaller group.

# What we learned at stop 4

- You can test a study design by simulation before collecting any data.
- Power is simply the share of simulated studies that find the real effect.
- The same trick checks whether any statistical method behaves as advertised.

Stop 5

# Schelling's neighbourhood, and Monte Carlo over an agent-based model

One run is an anecdote. Many runs are an experiment.

# The model: a city of particular households

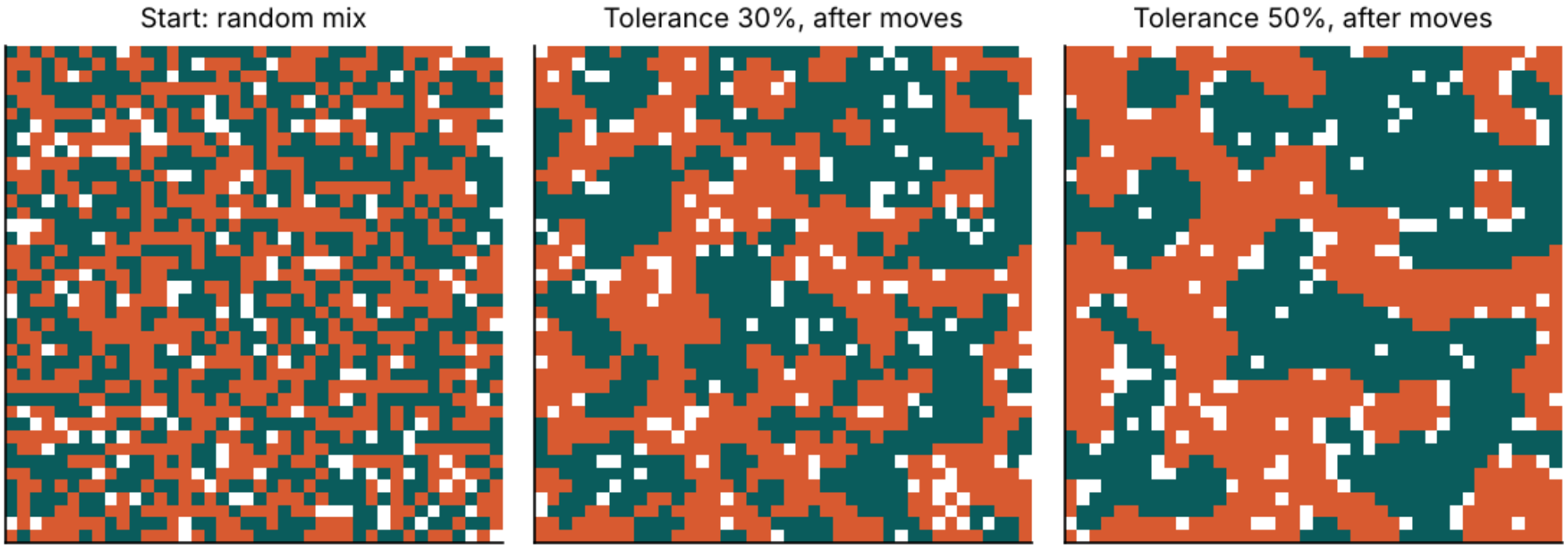
**Thomas C. Schelling**




- A grid of houses. Each is empty or holds a household of one of two kinds.
- Every household wants at least some share of its eight nearest neighbours to be its own kind: the tolerance threshold.
- Unhappy households move to a random empty house. Happy ones stay. That is the whole model.
- Schelling's question: how strong must the preference be before the city segregates? His answer: not strong at all.

*Schelling, T. C. (1971). Dynamic models of segregation. Journal of Mathematical Sociology, 1(2), 143–186. · Schelling (1978). Micromotives and Macrobehavior.*

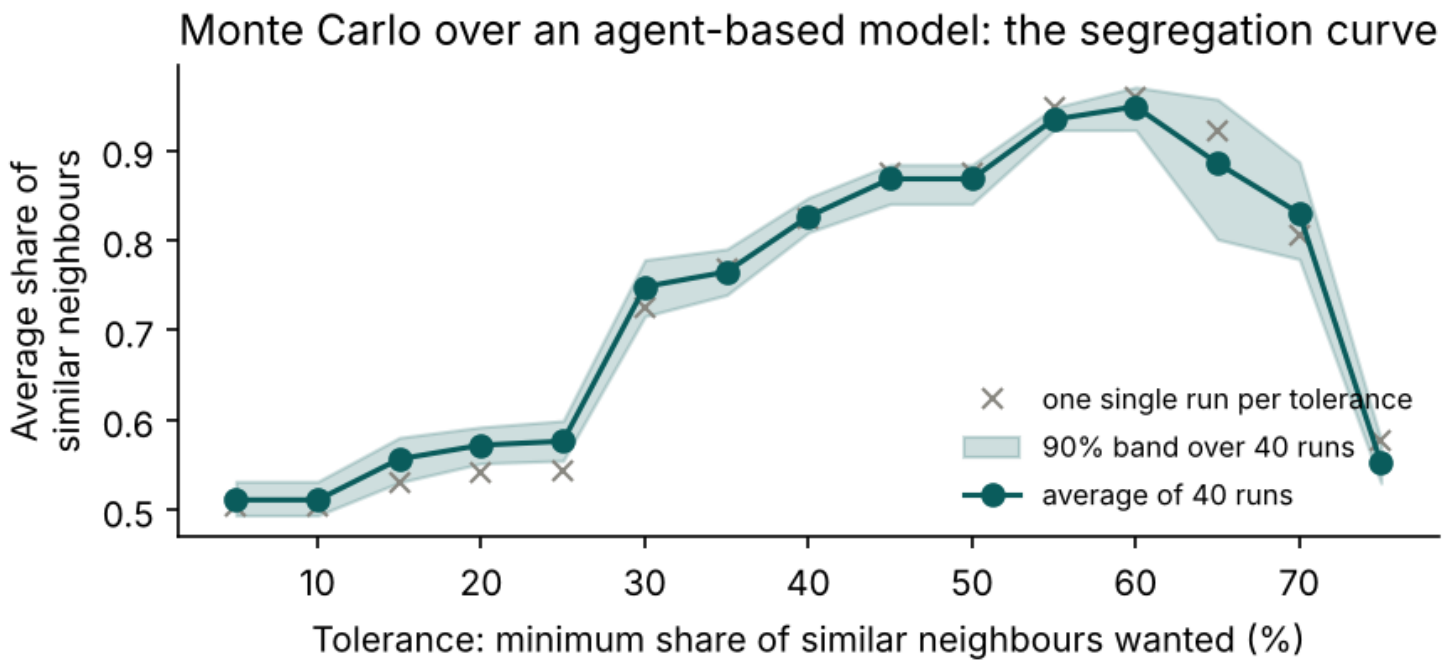
# One run of the model




*Left: the random starting city. Middle: after households with a 30% threshold have moved. Right: the same with a 50% threshold.*

**Run it again with a different seed and the clusters land elsewhere, with a somewhat different degree of segregation. Which one is “the result”? Neither.**

# Forty runs per threshold: the segregation curve




- A threshold: below 25% tolerance the city stays mixed; between 25% and 30% segregation jumps. With eight neighbours, that is the step from “two like me” to “three like me”.
- A surprise at the top: above about 65% almost nobody can be satisfied, households never stop moving, and the city never settles. Segregation falls again.
- A measure of confidence: the band shows the run-to-run variation to expect.

# Stop 5 · On your screen

**Website animation** “Schelling”: one run with sliders, then a Sweep button that runs many and draws the curve.

**Notebook** 5.1 defines the model, 5.2 one run, 5.3 the sweep (about a minute), 5.4 the preview of next seminar.

| Parameter | What it means | What to watch |
|---|---|---|
| **Tolerance** | Minimum share of similar neighbours a household wants | Clusters appear already at 30%; above 65% the city churns and never settles |
| **Seed** | Which random starting city and which random moves | Same tolerance, different seed: clusters elsewhere, segregation number differs |
| **Runs per point (sweep)** | How many seeds are averaged at each tolerance | More runs: smoother curve, narrower band |
| **Empty share** | Fraction of empty houses | More empties make moving easier: does the jump move? |

✎ **Exercise, for later (notebook):** repeat the sweep with a 15×15 city (the band widens: why?) and with 30% empty houses. Then try the Granovetter playground cell.

# The rule for every agent-based model

**Never report a single run. Run the model many times with different seeds, report the distribution of outcomes, and sweep the parameter you care about to find where the behaviour changes.**

*That sweep is a Monte Carlo experiment. It is what turns a simulation from a story into evidence.*

Playgrounds on the course website and in the notebook: Granovetter's threshold model of collective action (same average threshold, different spread, opposite outcome) and the forest-fire model (sweep tree density to find its threshold).

*Granovetter, M. (1978). Threshold models of collective behavior. American Journal of Sociology, 83(6), 1420–1443.*

# Looking ahead: where did 30% come from?

- Nowhere. It stands in for an attitude that, in a real city, you would measure with a questionnaire item: “I would feel uncomfortable if most of my neighbours were different from me.”
- Turning such answers into a number a simulation can use is called pseudo-quantification.
- Different people give different answers, so there is not one threshold but a distribution of thresholds. Carrying our doubt about the translation into the model is the quantification of uncertainty.
- Both are the subject of the next seminar, with the same model and one line of code changed: the single tolerance value becomes an array, one per household, drawn from survey data. Notebook 5.4 is the preview.

# Where this road leads

## Bootstrapping

A confidence interval for a statistic with no formula: resample your own survey with replacement, recompute, repeat 10,000 times. The spread is your interval. Monte Carlo on data you already have. Notebook 6.1.

## Sensitivity analysis

Real models have dozens of uncertain inputs. Shake them all at once, then ask which one explains most of the variation in the output. That is the one worth measuring precisely. Notebook 6.2.

## Markov chain Monte Carlo

Bayesian statistics needs samples from a distribution you cannot write down. MCMC takes a random walk that lingers where the data fit well. Engine of modern Bayesian software (Stan, PyMC). Notebook 6.3, twelve lines.

*Efron, B. (1979). Bootstrap methods. Ann. Statist., 7(1), 1–26. · Metropolis, N., et al. (1953). Equation of state calculations by fast computing machines. J. Chem. Phys., 21(6), 1087–1092.*

# Five habits that keep you honest

- Garbage in, gospel out. Ten thousand runs on a wrong assumption give a beautifully smooth histogram that is precisely wrong. State your input assumptions; test what happens when you change them.
- Report the spread, not just the average: a probability, a median, a 5th to 95th percentile range, a band around a curve.
- Use enough runs, and say how many. A thousand is the minimum; more for the tails. If two repetitions disagree visibly, you need more.
- Fix the seed and share the code, so anyone can reproduce your numbers exactly.
- A model is not the world. Monte Carlo quantifies the uncertainty inside a model; it cannot tell you whether the model is right.

# Now you: ten minutes of free play

## Website animations

acharyakishor.com/simulations
Dice, darts, referendum, power, Schelling, plus Granovetter and the forest fire. Each has a “Try this” box.

## Colab notebook

Link on the course page. Same numbering as these slides. Change any number, watch the figure redraw. No installation.

For later: the notebook has an ✎ Exercise at the end of every stop (doubles, darts precision, a fieldwork budget, an unequal design, a small city), and three extras that only live there: Buffon's needle, a coverage check, and a twelve-line MCMC sampler.

# References


Bernoulli, J. (1713). Ars Conjectandi. Basel: Thurneysen.

Buffon, G.-L. L., Comte de (1777). Essai d'arithmétique morale. Histoire naturelle, Supplément, vol. 4.

Cohen, J. (1988). Statistical Power Analysis for the Behavioral Sciences (2nd ed.). Lawrence Erlbaum.

Efron, B. (1979). Bootstrap methods: Another look at the jackknife. The Annals of Statistics, 7(1), 1–26.

Granovetter, M. (1978). Threshold models of collective behavior. American Journal of Sociology, 83(6), 1420–1443.

Metropolis, N. (1987). The beginning of the Monte Carlo method. Los Alamos Science, 15, 125–130.

Metropolis, N., & Ulam, S. (1949). The Monte Carlo method. Journal of the American Statistical Association, 44(247), 335–341.

Metropolis, N., Rosenbluth, A. W., Rosenbluth, M. N., Teller, A. H., & Teller, E. (1953). Equation of state calculations by fast computing machines. J. Chem. Phys., 21(6), 1087–1092.

Schelling, T. C. (1971). Dynamic models of segregation. Journal of Mathematical Sociology, 1(2), 143–186.

Schelling, T. C. (1978). Micromotives and Macrobehavior. W. W. Norton.

Verschuuren, G. M. (2017). 100 Excel Simulations. Holy Macro! Books.

# Thank you

Generate many random versions of the situation. Count what happens. Report the spread.